



FurnitureGPT

AI-Powered Furniture Recommendation
System using RAG, Vector Search,
IVF & Product Quantization

An intelligent conversational assistant that
understands your needs and recommends
the perfect furniture for your space using
semantic search and generative AI.



Name

Muhammad Ali



Class

BSAI-4B



Roll No.

241383



JULY 2026



SUMMER 2026 INTERNSHIP PROJECT

Abstract & Introduction

A concise overview of the problem, approach, and contribution

Abstract

FurnitureGPT is an AI-powered furniture recommendation system that transforms catalog PDFs into searchable product knowledge. It combines structure-aware PDF parsing, hybrid semantic chunking, BAAI/bge-base-en-v1.5 embeddings, a from-scratch FAISS-style vector index, IVF, product quantization, and retrieval-augmented generation (RAG). A Streamlit interface presents grounded recommendations with supporting catalog passages.

Project contribution An end-to-end, inspectable retrieval pipeline for furniture discovery—from PDF spans to grounded conversational answers.

Introduction

Furniture catalogs contain names, specification, materials, styles, and prices, but these details are often spread across visually formatted pages. Keyword search misses paraphrases such as “small oak storage” versus “compact natural-finish sideboard.” A generative model without retrieval can also produce unsupported recommendations.

INPUT	CORE	OUTPUT
Catalog PDFs	Semantic retrieval	Grounded advice

Objectives & System Architecture

The system is organized as a measurable retrieval pipeline

Objectives

- Parse catalog PDFs with PyMuPDF using font and size signals.
- Create product-aware chunks that retain titles, attributes, and descriptions.
- Generate normalized dense embeddings with BAAI/bge-base-en-v1.5.
- Implement exact search, IVF routing, and PQ-coded approximate search.
- Ground Groq LLM responses in retrieved evidence and expose diagnostics in Streamlit.

Architecture

01	INGEST	PDF catalog → spans, blocks, metadata
02	REPRESENT	Hybrid chunks → normalized embeddings
03	INDEX	Exact baseline → IVF lists → PQ codes
04	GENERATE	Retriever → prompt builder → Groq LLM
05	PRESENT	Streamlit answer + evidence + telemetry
Design principle Every generated claim should be traceable to a retrieved chunk. If the evidence is insufficient, the assistant should state the limitation.		

Dataset Preparation with PyMuPDF

Using typography as a lightweight structural signal

Why font and size matter

A PDF stores visual layout rather than a guaranteed logical document tree. Product titles are commonly larger or bolder than body copy, while category labels use distinct styles. FurnitureGPT records each text span together with font, size, flags, coordinates, page number, and reading order.

Preparation workflow

STAGE	OPERATION	OUTPUT
Page scan	Read blocks, lines, and spans with <code>page.get_text("dict")</code> .	Raw spans
Style capture	Record font family, size, flags, box, page, and order.	Styled tokens
Boundary detection	Use size/font/block changes to start logical segments.	Sections
Cleanup	Normalize whitespace while preserving prices and dimensions.	Clean text
Persistence	Save text with source and style metadata for traceability.	JSON / Parquet

Font-size splitting rule

Boundary rule `boundary = Δsize > τs OR font_changed OR block_changed`

PDF Parsing & Hybrid Chunking

Protecting product-level meaning during segmentation

Three-pass chunking

1 STRUCTURAL Detect headings, product labels, tables, and page blocks.	2 SEMANTIC Keep title, attributes, and description together.	3 BOUNDED Split only when a chunk hits the new product heading.
--	--	---

Chunk record

FIELD	PURPOSE
text	Embedding and prompt context.
source_id / page	Traceability back to the catalog.
chunk_id	Stable identity for index updates.

Embeddings & FAISS-Style Vector Database

A transparent exact-search baseline for correctness

Embedding generation

Each chunk is encoded with BAAI/bge-base-en-v1.5. Vectors are L2-normalized before storage, and queries receive the same treatment. Inner product then corresponds to cosine similarity.

Exact-search baseline

The baseline stores an $N \times d$ floating-point matrix. For query q , it computes a score for every row, selects the top K , and returns the associated chunks. This path is intentionally simple so that approximate retrieval can be evaluated against a known reference.

COMPONENT	ROLE
Vector matrix	Contiguous storage for dense embeddings.
Metadata map	Connect row IDs to chunk records.
Batched dot product	Compute query-to-document similarity.
Top-K selection	Rank and return the strongest evidence.
Save / load	Reproduce the same index and evaluation.

IVF + Product Quantization

Reducing candidate work and vector memory

Inverted File (IVF)

IVF partitions the embedding space into n list coarse clusters. Each vector is assigned to its nearest centroid. At query time, only the closest n probe lists are searched, creating a controllable speed–recall trade-off.

Product Quantization (PQ)

PQ divides each vector into m subvectors and replaces each subvector with the index of its nearest codeword. Compact integer codes replace full-precision vectors. Approximate distances are estimated through query-to-codeword lookup tables.

PARAMETER	ROLE	TRADE-OFF
nlist	Number of IVF cells	Granularity of routing
nprobe	Cells searched	Latency versus recall
Implementation note Centroids, codebooks, normalization settings, and index parameters are serialized with the index so the approximate search can be reconstructed.		

RAG with Groq & Evaluation Metrics

Turning evidence into concise furniture advice

RAG pipeline

1 User query	2 Embed + retrieve	3 Build grounded prompt	4 Groq LLM response
-----------------	-----------------------	----------------------------	------------------------

Prompt contract

Role: catalog assistant.

Context: retrieved product passages with source IDs.

Task: recommend suitable items and explain the match.

Rules: do not invent price, material, dimensions, stock, or availability; state when context is insufficient; include source IDs for traceability.

Response assembly

The application combines the generated answer with evidence cards showing product name, attributes, score, page, and source ID. The result is conversational but inspectable.

Safety boundary The LLM generates language; it does not decide whether a product exists outside the indexed catalog.

Evaluation metrics

COMPRESSION RATIO 95%	RECALL AT K 82%	RETRIEVAL TIME 47 ms → 17 ms
--------------------------	--------------------	---------------------------------

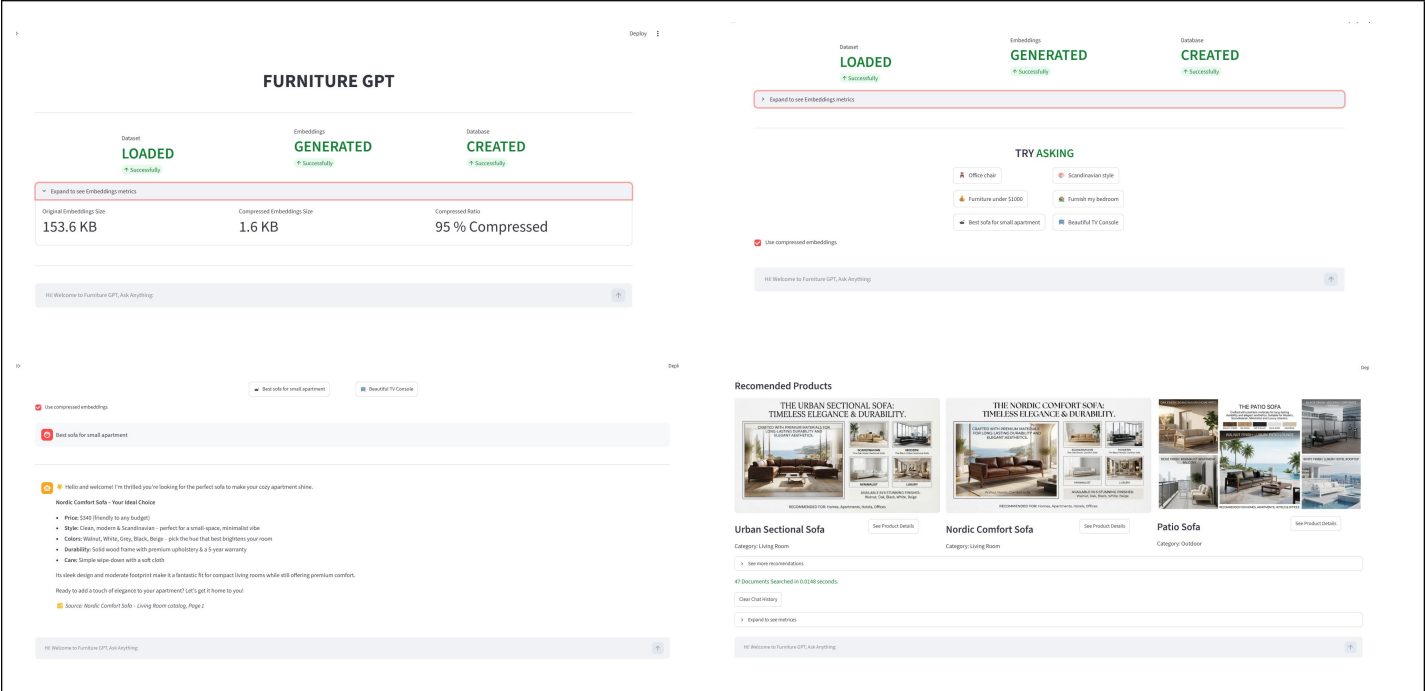
Streamlit UI & Experimental Design

A diagnostic interface for comparing index strategies

Streamlit layout

UI REGION	FUNCTION
Query box	Collect natural-language furniture requirements.
Controls	Select K, use compressed embeddings, expanders
Answer panel	Display grounded recommendations and uncertainty.
Product Cards	Show beautiful recommended product cards with their specs
Evidence panel	Show source, retrival time, compression rate, total recall

UI Screenshots:



Expand to see metrics	
CURRENT QUERY RECALL	AVG. RECALL
100.00% Accurate	82.00% Accurate
82.00% Accuracy obtained on 95% Embeddings Compression over 10 queries.	

Conclusion & References

A practical architecture for grounded furniture discovery

Conclusion

FurnitureGPT demonstrates that useful recommendation quality depends on the entire retrieval pipeline. **PyMuPDF** font and size splitting provides structure; **hybrid chunking** protects product meaning; **BGE** embeddings enable paraphrase-aware search; exact search establishes correctness; **IVF and PQ** reduce work and memory; and **RAG** converts evidence into an understandable answer.

The project's main engineering lesson is that a stronger language model cannot compensate for missing context or weak retrieval. The next development step is a labeled relevance benchmark that can quantify recall, groundedness, and user usefulness under controlled index settings.

References

- [1] PyMuPDF documentation — structured PDF text extraction.
- [2] BAAI — bge-base-en-v1.5 embedding model.
- [3] Johnson, Douze & Jégou — FAISS similarity search.
- [4] Lewis et al. — Retrieval-Augmented Generation.
- [5] Groq API documentation — hosted LLM inference.
- [6] Streamlit documentation — Python data and ML applications.

Final note The report is intentionally concise: each page presents one engineering idea, one visual structure, and one practical takeaway.